

rounds a positive (or unsigned) 32-bit integer v up to the next power of 2 that is $\geq v$.

When we use the bit-twiddling hacks, we'll include an explanatory comment in the code.

1.2.4 Utility Macros or Templated Functions

The file `nr3.h` includes, among other things, definitions for the functions

```
MAX(a,b)
MIN(a,b)
SWAP(a,b)
SIGN(a,b)
```

These are all self-explanatory, except possibly the last. `SIGN(a,b)` returns a value with the same magnitude as `a` and the same sign as `b`. These functions are all implemented as templated inline functions, so that they can be used for all argument types that make sense semantically. Implementation as macros is also possible.

CITED REFERENCES AND FURTHER READING:

Harbison, S.P., and Steele, G.L., Jr. 2002, *C: A Reference Manual*, 5th ed. (Englewood Cliffs, NJ: Prentice-Hall).[1]
 Anderson, S.E. 2006, "Bit Twiddling Hacks," at <http://graphics.stanford.edu/~seander/bithacks.html>. [2]

1.3 Objects, Classes, and Inheritance

An *object* or *class* (the terms are interchangeable) is a program structure that groups together some variables, or functions, or both, in such a way that all the included variables or functions "see" each other and can interact intimately, while most of this internal structure is hidden from other program structures and units. Objects make possible *object-oriented programming* (OOP), which has become recognized as the almost unique successful paradigm for creating complex software. The key insight in OOP is that objects have *state* and *behavior*. The state of the object is described by the values stored in its member variables, while the possible behavior is determined by the member functions. We will use objects in other ways as well.

The terminology surrounding OOP can be confusing. Objects, classes, and structures pretty much refer to the same thing. Member functions in a class are often referred to as *methods* belonging to that class. In C++, objects are defined with either the keyword `class` or the keyword `struct`. These differ, however, in the details of how rigorously they hide the object's internals from public view. Specifically,

```
struct SomeName { ...
```

is defined as being the same as

```
class SomeName {
public: ...
```

In this book we *always* use `struct`. This is not because we deprecate the use of `public` and `private` access specifiers in OOP, but only because such access control would add little to understanding the underlying numerical methods that are the focus of this book. In fact, access specifiers could impede your understanding, because

you would be constantly moving things from private to public (and back again) as you program different test cases and want to examine different internal, normally private, variables.

Because our classes are declared by `struct`, not `class`, use of the word “class” is potentially confusing, and we will usually try to avoid it. So “object” means `struct`, which is really a class!

If you are an OOP beginner, it is important to understand the distinction between defining an object and instantiating it. You define an object by writing code like this:

```
struct Twovar {
    Doub a,b;
    Twovar(const Doub aa, const Doub bb) : a(aa), b(bb) {}
    Doub sum() {return a+b;}
    Doub diff() {return a-b;}
};
```

This code does not create a `Twovar` object. It only tells the compiler how to create one when, later in your program, you tell it to do so, for example by a declaration like,

```
Twovar mytwovar(3.,5.);
```

which invokes the `Twovar` constructor and creates an instance of (or *instantiates*) a `Twovar`. In this example, the constructor also sets the internal variables `a` and `b` to 3 and 5, respectively. You can have any number of simultaneously existing, noninteracting, instances:

```
Twovar anothertwovar(4.,6.);
Twovar athirdtwovar(7.,8.);
```

We have already promised you that this book is not a textbook in OOP, or the C++ language; so we will go no farther here. If you need more, good references are [1-4].

1.3.1 Simple Uses of Objects

We use objects in various ways, ranging from trivial to quite complex, depending on the needs of the specific numerical method that is being discussed. As mentioned in §1.0, this lack of consistency means that *Numerical Recipes* is not a useful exemplar of a program library (or, in an OOP context, a *class library*). It also means that, somewhere in this book, you can probably find an example of every possible way to think about objects in numerical computing! (We hope that you will find this a plus.)

Object for Grouping Functions. Sometimes an object just collects together a group of closely related functions, not too differently from the way that you might use a namespace. For example, a simplification of Chapter 6’s object `Erf` looks like:

```
struct Erf {
    Doub erf(Doub x);
    Doub erfc(Doub x);
    Doub inverf(Doub p);
    Doub inverfc(Doub p);
    Doub erfccheb(Doub z);
};
```

No constructor needed.

As will be explained in §6.2, the first four methods are the ones intended to be called by the user, giving the error function, complementary error function, and the two

corresponding inverse functions. But these methods share some code and also use common code in the last method, `erfccheb`, which the user will normally ignore completely. It therefore makes sense to group the whole collection as an `Erf` object. About the only disadvantage of this is that you must instantiate an `Erf` object before you can use (say) the `erf` function:

```
Erf myerf;                      The name myerf is arbitrary.
...
Doub y = myerf.erf(3.);
```

Instantiating the object doesn't actually *do* anything here, because `Erf` contains no variables (i.e., has no stored state). It just tells the compiler what local name you are going to use in referring to its member functions. (We would normally use the name `erf` for the instance of `Erf`, but we thought that `erf.erf(3.)` would be confusing in the above example.)

Object for Standardizing an Interface. In §6.14 we'll discuss a number of useful standard probability distributions, for example, normal, Cauchy, binomial, Poisson, etc. Each gets its own object definition, for example,

```
struct Cauchydist {
    Doub mu, sig;
    Cauchydist(Doub mmu = 0., Doub ssig = 1.) : mu(mmu), sig(ssig) {}
    Doub p(Doub x);
    Doub cdf(Doub x);
    Doub invcdf(Doub p);
};
```

where the function `p` returns the probability density, the function `cdf` returns the cumulative distribution function (cdf), and the function `invcdf` returns the inverse of the cdf. Because the interface is consistent across all the different probability distributions, you can change which distribution a program is using by changing a single program line, for example from

```
Cauchydist mydist();
```

to

```
Normaldist mydist();
```

All subsequent references to functions like `mydist.p`, `mydist.cdf`, and so on, are thus changed automatically. This is hardly OOP at all, but it can be very convenient.

Object for Returning Multiple Values. It often happens that a function computes more than one useful quantity, but you don't know which one or ones the user is actually interested in on that particular function call. A convenient use of objects is to save all the potentially useful results and then let the user grab those that are of interest. For example, a simplified version of the `Fitab` structure in Chapter 15, which fits a straight line $y = a + bx$ to a set of data points `xx` and `yy`, looks like this:

```
struct Fitab {
    Doub a, b;
    Fitab(const VecDoub &xx, const VecDoub &yy);    Constructor.
};
```

(We'll discuss `VecDoub` and related matters below, in §1.4.) The user calculates the fit by calling the constructor with the data points as arguments,

```
Fitab myfit(xx,yy);
```

Then the two “answers” a and b are separately available as `myfit.a` and `myfit.b`. We will see more elaborate examples throughout the book.

Objects That Save Internal State for Multiple Uses. This is classic OOP, worthy of the name. A good example is Chapter 2’s `LUdcmp` object, which (in abbreviated form) looks like this:

```
struct LUDcmp {
    Int n;
    MatDoub lu;
    LUDcmp(const MatDoub &a); Constructor.
    void solve(const VecDoub &b, VecDoub &x);
    void inverse(MatDoub &ainv);
    Doub det();
};
```

This object is used to solve linear equations and/or invert a matrix. You use it by creating an instance with your matrix a as the argument in the constructor. The constructor then computes and stores, in the internal matrix `lu`, a so-called LU decomposition of your matrix (see §2.3). Normally you won’t use the matrix `lu` directly (though you could if you wanted to). Rather, you now have available the methods `solve()`, which returns a solution vector x for any right-hand side b , `inverse()`, which returns the inverse matrix, and `det()`, which returns the determinant of your matrix.

You can call any or all of `LUdcmp`’s methods in any order; you might well want to call `solve` multiple times, with different right-hand sides. If you have more than one matrix in your problem, you create a separate instance of `LUdcmp` for each one, for example,

```
LUdcmp alu(a), aalu(aa);
```

after which `alu.solve()` and `aalu.solve()` are the methods for solving linear equations for each respective matrix, a and aa ; `alu.det()` and `aalu.det()` return the two determinants; and so forth.

We are not finished listing ways to use objects: Several more are discussed in the next few sections.

1.3.2 Scope Rules and Object Destruction

This last example, `LUdcmp`, raises the important issue of how to manage an object’s time and memory usage within your program.

For a large matrix, the `LUdcmp` constructor does a lot of computation. You choose exactly where in your program you want this to occur in the obvious way, by putting the declaration

```
LUdcmp alu(a);
```

in just that place. The important distinction between a non-OOP language (like C) and an OOP language (like C++) is that, in the latter, declarations are not passive instructions to the compiler, but executable statements at run-time.

The `LUdcmp` constructor also, for a large matrix, grabs a lot of memory, to store the matrix `lu`. How do you take charge of this? That is, how do you communicate that it should save this state for as long as you might need it for calls to methods like `alu.solve()`, but not indefinitely?

The answer lies in C++’s strict and predictable rules about *scope*. You can start a temporary scope at any point by writing an open bracket, “{”. You end that scope by a matching close bracket, “}”. You can nest scopes in the obvious way. Any objects that are declared within a scope are destroyed (and their memory resources returned) when the end of the scope is reached. An example might look like this:

<code>MatDoub a(1000,1000);</code>	Create a big matrix,
<code>VecDoub b(1000),x(1000);</code>	and a couple of vectors.
<code>...</code>	
<code>{</code>	Begin temporary scope.
<code>LUdcmp alu(a);</code>	Create object alu.
<code>...</code>	
<code>alu.solve(b,x);</code>	Use alu.
<code>...</code>	
<code>}</code>	End temporary scope. Resources in alu are freed.
<code>...</code>	
<code>Doub d = alu.det();</code>	ERROR! alu is out of scope.

This example presumes that you have some other use for the matrix `a` later on. If not, then the declaration of `a` should itself probably be inside the temporary scope.

Be aware that *all* program blocks delineated by braces are scope units. This includes the main block associated with a function definition and also blocks associated with control structures. In code like this,

```
for (;;) {
    ...
    LUdcmp alu(a);
    ...
}
```

a new instance of `alu` is created at each iteration and then destroyed at the end of that iteration. This might sometimes be what you intend (if the matrix `a` changes on each iteration, for example); but you should be careful not to let it happen unintentionally.

1.3.3 Functions and Functors

Many routines in this book take functions as input. For example, the quadrature (integration) routines in Chapter 4 take as input the function $f(x)$ to be integrated. For a simple case like $f(x) = x^2$, you code such a function simply as

```
Doub f(const Doub x) {
    return x*x;
}
```

and pass `f` as an argument to the routine. However, it is often useful to use a more general object to communicate the function to the routine. For example, $f(x)$ may depend on other variables or parameters that need to be communicated from the calling program. Or the computation of $f(x)$ may be associated with other subcalculations or information from other parts of the program. In non-OOP programming, this communication is usually accomplished with global variables that pass the information “over the head” of the routine that receives the function argument `f`.

C++ provides a better and more elegant solution: *function objects* or *functors*. A functor is simply an object in which the operator `()` has been overloaded to play the role of returning a function value. (There is no relation between this use of the word functor and its different meaning in pure mathematics.) The case $f(x) = x^2$ would now be coded as